

ListMerger: A JavaScript Component for Manual List Merging

Martin Rabensteiner

706.414 Seminar/Project Interactive and Visual Information Systems 4SP WS 2025/2026
Graz University of Technology

24 Jun 2026

Abstract

ListMerger is a TypeScript-written JavaScript library to facilitate the merging of multiple lists. An application example is the merging of findings from multiple evaluators in a heuristic evaluation. The library bases primarily on modern usage of HTML, CSS, and JavaScript, and largely avoids third-party plugins. This paper contains a description of the implementation, a brief introduction into the techniques and use cases, and user guides for developers, implementors, and end-users.

© Copyright 2026 by the author, except as otherwise noted.

This work is placed under a Creative Commons Attribution 4.0 International (CC BY 4.0) licence.

Contents

Contents	ii
List of Figures	iii
List of Tables	v
List of Listings	vii
1 Introduction	1
2 List Merging	3
2.1 Acquisition Lists	3
2.2 Recipes	3
2.3 Heuristic Evaluations	3
2.4 Related Work	4
3 Modern Web Technologies	7
3.1 Living HTML.	7
3.2 Modern CSS	7
3.3 Modern JavaScript	7
3.4 jQuery	8
3.5 JavaScript Package Managers	8
3.6 Mustache Templates	8
4 ListMerger	9
4.1 Architecture and Files	9
4.2 Initialisation and Data Structure	9
4.3 History (Undo/Redo)	10
4.4 Tab Bar	10
4.5 Items	12
4.6 User Actions and their implementations	14
4.6.1 Moving Items	14
4.6.2 Editing Items	14
4.6.3 Merging Items	14
4.6.4 Detaching Items	14

4.7	Icons	14
4.8	Challenges and Future Work	14
4.8.1	Cursor Behaviour when Dragging.	14
4.8.2	Flexbox Layout within display: contents	15
4.8.3	Accessibility	15
A	End User Guide	17
A.1	Functionality	17
A.2	Keyboard Shortcuts.	17
B	Library User Guide	19
B.1	Examples	19
B.2	Initialising	19
B.3	Data Structure.	19
B.4	Templating	20
B.5	Selectors	21
B.6	Saving	21
B.7	Inline Editing	21
B.8	Draghandle.	23
	Bibliography	25

List of Figures

2.1	Sketch of a Recipe Overview	4
2.2	HEM Findings Merger	5
2.3	Heurio: Kanban board	5
4.1	Tabs	11
4.2	Tabs Select Box	11
4.3	Item Editing	12
4.4	Merge Item Window	13
4.5	Merged Item	13
A.1	ListMerger Example	18
B.1	Inline Editing	23

List of Tables

4.1	Icons used in ListMerger	15
B.1	HTML Identifiers	22

List of Listings

4.1	Tab Bar CSS	10
B.1	JSON Item Lists	20

Chapter 1

Introduction

This report describes the purpose, development, and usage of a tool for list merging. The use cases are further described in Chapter 2 and the development in Chapter 4. Finally, **[TODO:]**.

Chapter 2

List Merging

List merging as a term describes the process to merge the items of several lists into one aggregated list. This chapter covers use cases and the affordance of list merging. Further, implementation examples and related work is described.

2.1 Acquisition Lists

Different departments or employees can enter their needs and wishes for supplies. An acquisition manager has then an overview of all wishes. Identical or similar items can be combined and the quantity of the concerning item increased.

2.2 Recipes

Recipes would be use case where similar items in different lists would be beneficial and desirable. The origin lists are for recipes selected by the prospective cook. In the merged list, the needed or the already available ingredients can be gathered. This highlights overlapping ingredients in recipes and helps to depict which recipes can be cooked with the available ones or which ones need to be bought. A sketch of this example can be seen in Figure 2.1.

2.3 Heuristic Evaluations

The heuristic evaluation is a form of usability inspection, first described by Nielsen and Molich [1990]. A small number of usability experts, usually 3 to 5, review a user interface and use their judgement and experience to find potential problems. The evaluation is guided by a small set of broad principles (the heuristics) for good user interface design. Outside academic circles, the technique is often known as an *expert review*. Andrews [2025] describes the ideal process of a heuristic evaluation with five phases: Preparation, Individual Evaluations, Aggregation, Severity Ratings, and Debriefing and Report. The essential steps of list merging are the second and third. In the Individual Evaluations phase, the evaluators observe the interface completely independent and described the issues found by them. In the Aggregation phase afterwards, an evaluation manager needs to aggregate these findings and combine similar ones. Therefore, the evaluation manager must be able to simple combine the issues with all their metadata (heuristic, description, attached screenshots, and so on) Those are then further rated according to their severity and described in a report.

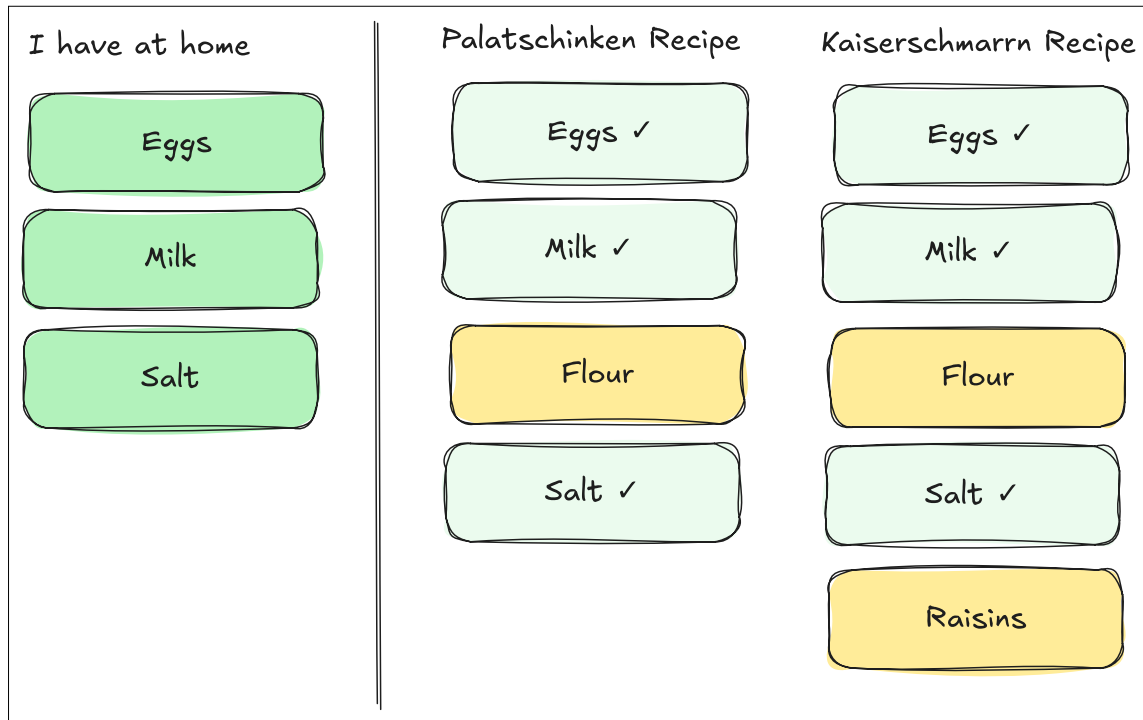


Figure 2.1: In this example, the ingredients of recipes that are already to hand can be put on the merge list, to emphasise ingredients that need to be bought first. [Sketch by Martin Rabensteiner using Excalidraw.]

2.4 Related Work

For the last mentioned application case, Loitzl [2006] introduces within the Heuristic Evaluation Manager (HEM) a merging interface to merge usability issues, as seen in Figure 2.2. Findings can be selected for merging with the + button. Red indicates currently selected findings. Darkgrey findings are already added to the merge list. This is done with inline JavaScript calls by pressing a button, and without drag-and-drop functions.

The functionality of Kanban boards, like the issue tracking app Heurio in Figure 2.3 seems similar, but the columns represent different stages or filters, and the possibility to merge is not designated.

By researching, no equivalent library, nor applications with dynamic, visual merging could be found. Therefore it was decided to create ListMerger with its implementation and functionality further described in the following chapters.

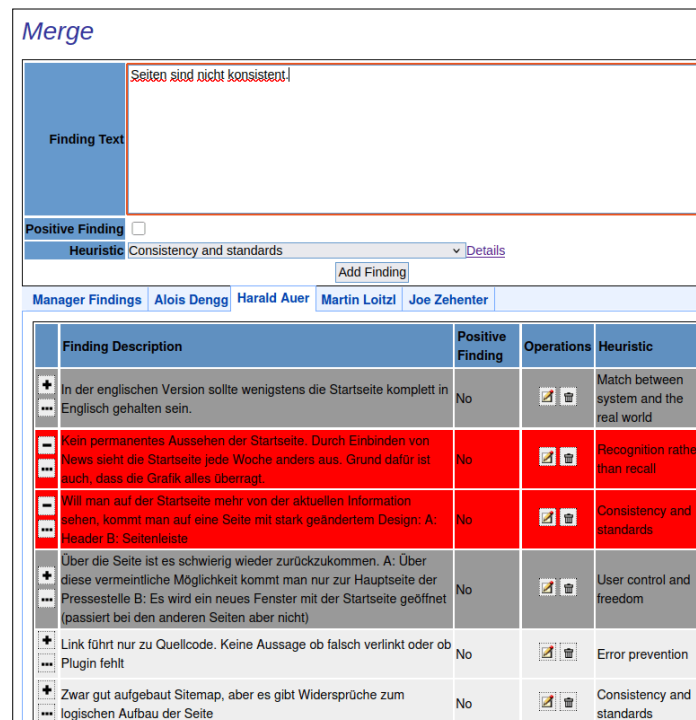


Figure 2.2: The findings merger in HEM: Findings can be selected for merging with the + button. Red indicates currently selected findings. Darkgrey findings are already added to the merge list. [Screenshot by Martin Rabensteiner.]

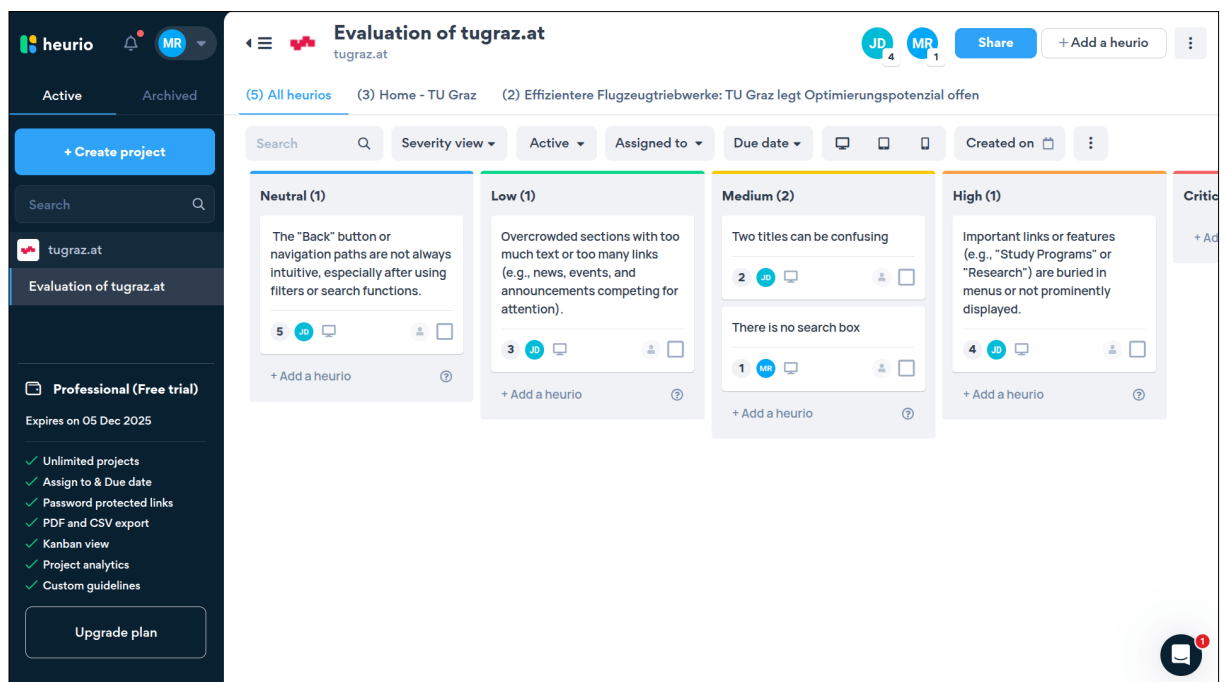


Figure 2.3: Usability findings in Heurio are by default organised as a Kanban board and sorted by severity. [Screenshot by Martin Rabensteiner.]

Chapter 3

Modern Web Technologies

Web technologies are a steadily changing field to keep track with new technical possibilities and optical or interactivity affordances. This chapter covers some topics related to the development of ListMerger.

The main browser vendors united in 2004 as Web Hypertext Application Technology Working Group (WHATWG), but the definition of the HTML standard was in charge of the World Wide Web Consortium (W3C) for some decades. In 2018, the browser vendors became mutinous and only one year later, both signed an agreement where W3C is giving the vendors more say [Cimpanu 2019]. This led to more standardization among different browsers, and a faster drive of innovation, like the improvements mentioned as follows.

3.1 Living HTML

The Hyper Text Markup Language (HTML) standard was first developed and published in the early 1990's by Tim Berners-Lee at CERN and later handed over to the new founded World Wide Web Consortium (W3C) [WHATWG 2026]. The last major version (HTML4) was published in 1998, and after some intermediate versions, the work on HTML5 started by W3C in 2004. HTML5 was never published in a finished version. Since 2019, W3C and WHATWG are maintaining the standard together as a living standard, and publish updates continuously in the publishing browsers.

3.2 Modern CSS

Along with the HTML modernisation, the Cascading Style Sheets (CSS) became a living standard as well [W3C 2026]. CSS is used to define the optical style of a website. Modern solutions include for example containers queries [MDN 2026b] to differentiate between different screen sizes or use styles as selectors, or colour mixing functions [MDN 2026a].

3.3 Modern JavaScript

JavaScript is essential in modern web development for interactivity [Shukla 2023]. It enables to make client-site computations and modify the Document Object Model (DOM), which describes the structure and content of a web page. It was introduced in 1995, soon available in all usual browsers, and is and still today an important component of them. As dynamic web pages become more and more popular, it was even more necessary to manipulate the content directly on the client. With Node.js, first published in 2009, it became more and more popular to use JavaScript also as a server-side programming language [Node.js 2026]. JavaScript can be therefore used in the backend as well as in the frontend, creating new synergies.

3.4 jQuery

jQuery is a widely-used frontend framework and reached its peak of usage on around 78 % of all websites tracked in 2022, but is steadily decreasing since then, with circa 70 % in March 2026 [W3Techs 2026]. By time, the outstanding functions of jQuery got more and more redundant. Several use cases and functionalities were integrated directly in HTML, CSS, or, mostly, JavaScript. For example, the accordion can now be replaced with the `<details>` tag (available since 2020) [MDN 2025a]. JavaScript introduced selector functions like `querySelector()`, `querySelectorAll()` and some other for HTML element in 2015 [MDN 2025c]. This made the classic jQuery DOM selection with the `\$()` function obsolete.

3.5 JavaScript Package Managers

The node package manager (NPM) was introduced in 2009, to make the handling with node modules in the development process and their publishment among developers more convenient. It is an open source project and now in the hands of GitHub [NPM 2026].

PNPM (performant NPM) introduces a more performant approach for package managing than NPM [PNPM 2026]. Instead of downloading and storing dependencies for every project on its own, they are managed globally on the system, and symbolic links are created from the project to the main storage. That saves time and storage space. Further, PNPM is capable of managing different versions of Node. For other package managers, a separate Node Version Manager is needed. Disadvantages arise when multiple disks are used on a system, as the symlinks used by PNPM are not capable of that.

To measure the popularity of package managers, Maj [2024] discusses different ways to count the usage. Different ways of downloads have the problem of different sources, and packager managers with more frequent updates will have higher download counts. The observation of the most popular GitHub repositories is considered a meaningful measurement. According to the last iteration Maj 2026 and compared with the last three years, NPM is steadily used in about two third of projects. The usage of Yarn classic has decreased from 29 % to 18 % over the same time frame. PNPM has a stagnant share of about 6 %.

Another currently uprising package manager is bun, which claims to install dependencies a few times faster than the tools mentioned before [Bun 2026]. Its toolkit provides alternatives for Node.js, Vite, and other JavaScript development tools as well.

3.6 Mustache Templates

Mustache is a logic-less template format [Mustache 2024]. Characteristic are the double `{{}}` brackets, which are then replaced. For example, `{{variable}}` is replaced with the value of `variable`. For-each loops and simple if conditions can be mocked with Mustache, even though there is no logic in the template. Mustache libraries are available for around 50 programming languages, including JavaScript, Python, PHP, or C++. It is popular to structure small or large parts of content with its template style.

Chapter 4

ListMerger

This section describes implementation details of ListMerger. The code itself is written in pure TypeScript. Almost all third-party libraries are used only in the build and development process. Only the Mustache package is used to render templates during runtime.

Recurring terms in the development are move and merge. Moving describes the process of a transferring an item from an origin list to the mergelist. A merged item always consists of two predecessors. They are either taken from the mergelist (merged or moved), or directly taken from the evaluators list. The origin is always tracked, independent whether the item is coming from an origin list or the mergelist. The differentiation between moved and origin is done, to have a clean and reconstructive merging history.

4.1 Architecture and Files

The library itself is modularised into several TypeScript files and compiled with Gulp to a single .js file. Minimised and Gzip files, as well as a CSS stylesheet are added. This stylesheet contains the recommended minimum styling to build a proper example. The only dependency at runtime is Mustache [Mustache 2024] to render user-specific templates. The TypeScript files are:

- `events.ts`: Initialises all event listeners for buttons and drag events, and contains their functions.
- `globalvars.ts`: Contains some shared variables, all above the data structure for lists and items during runtime.
- `history.ts`: Tracks all actions and recovers them when the Undo or Redo button is pressed.
- `index.ts`: Serves as the interface between the module and the outer implementation.
- `init.ts`: Contains the only function callable from the outside.
- `transfer.ts`: Handles the transfers between the source lists and the merged list.
- `uihelper.ts`: Contains several helper functions to deal with the DOM.

4.2 Initialisation and Data Structure

An initialisation call is necessary to activate the functionality of the library. For this, the function `init.init()` is called from the outside. More details on this call are described in Section B.2. This function calls all necessary initialising functions of all files.

When initialising, a JavaScript object has to be handed over to the library in order to build the application. This object needs to contain at least a `merged` key and an `originlists` key. More details on

```
1 .container {  
2   display: flex;  
3   flex-wrap: wrap;  
4 }  
5  
6 details {  
7   display: contents;  
8 }  
9  
10 details[open]::details-content {  
11   display: contents;  
12 }  
13  
14 summary {  
15   order: 0;  
16 }  
17  
18 section {  
19   order: 1;  
20   width: 100%;  
21 }
```

Listing 4.1: The minimal CSS styling to use the `<details>` element and its children as a tab bar.

this data structure can be found in Section B.3, with Listing B.1 as example. The object is modified with every user action and is returned with the `getAllItems()` function.

ListMerger itself does not define its own style, but certain CSS styling rules are highly recommended and can be found in `listmerger.css`.

4.3 History (Undo/Redo)

An important feature and affordance of this component is the history tracking, to keep track of user actions. With the Undo and Redo buttons, the user can jump back and forward with actions.

The history is based on a history and a future array. When clicking undo/redo, a history item is moved from history to future, or vice versa. A history item consists of a keyword for the action, and several parameters, to track IDs, positions, arrays and so on, depending on the context of the merge keyword. User actions are tracked with the `history.log()` function.

4.4 Tab Bar

For a long time, it was necessary to use JavaScript event listeners for tab bars in the web. By adding click event listeners to the title, the appearance of the content below in that way can be controlled. The `<details>` HTML element and some CSS adjustments make it possible to have tab bars without JavaScript.

The `<details>` element serves as a container for one item. On `<summary>` element is needed as a child, which represents the title. If no such title element is present, the default is Details. Per default, a small triangle on the left indicates the open/closed status of the content. This status is toggled by clicking on the `<summary>` element. The content does not necessarily have to be an element and can also be plain text. To have a clear structure, it is still advisable to have an element to give the content a better structure, for

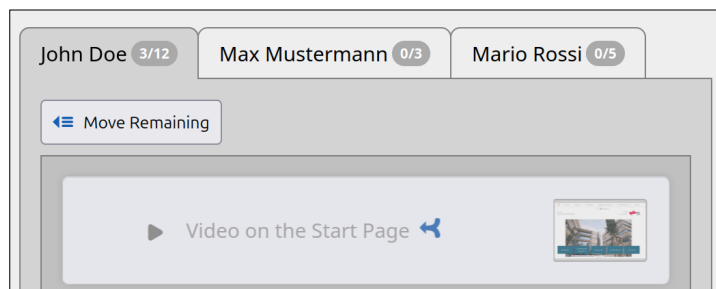


Figure 4.1: The tabs in ListMerger show the title of each list and an indicator with the number of added/merged and total items in the list. [Screenshot by Martin Rabensteiner.]

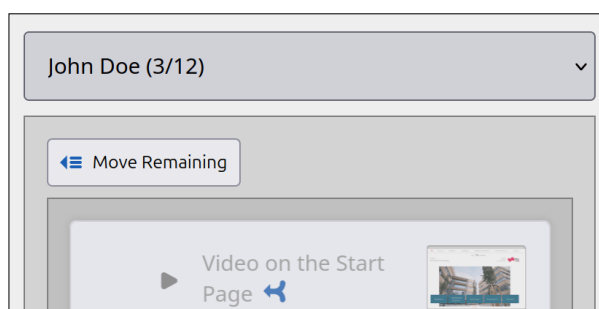


Figure 4.2: With less space, tabs are condensed into a select box. [Screenshot by Martin Rabensteiner.]

example with a `<section>` element. With the `open` attribute, the fold out of `<details>` can be controlled. The `name` attribute can be used to only open one `<details>` box when having several instances.

The original purpose of this element is to build Accordions. To use it for tab bars, some CSS properties need to be applied. These properties include:

- *`<container>` as `flexbox`*: This ensure to have the elements next to each in horizontal alignment.
- *`<section>` with `full-width (100%)`*: Below the flex container, the summaries should have only as much width as they need, but the section below should span across the whole width to break in a new line.
- *Display details as content*: The section would be shown right next to the summary, but this display option shifts it to the end.
- *Display open details pseudo-element as content*: This pseudo element
- *Order of summary and section*: Setting the order for all summary items to 0 and all section items to 1, clusters the summary items all at the top and shows the active section element below.

The minimal CSS to reach this appearance is shown in Listing 4.1. Additionally, the tabs are also styled with some basic CSS, including, colours, borders, border rounding, margins, paddings, and the cursor to reach a familiar visual appearance, as seen in Figure 4.1.

However, to maintain responsiveness the solution in ListMerger still uses JavaScript to transform the bar of tabs into a selection box on smaller screens, or with longer titles. A `<select>` box holds the titles as visible text and the order as values, as seen in Figure 4.2. The value of the box needs to be tracked and updated as well when the open tab changes. It also needs to update when elements are called with a hash in the URL.

A `ResizeObserver` instance helps to track the size or position of an element, changed for example through scrolling or resizing the window [MDN 2025f]. In this case it is used to observe the position of

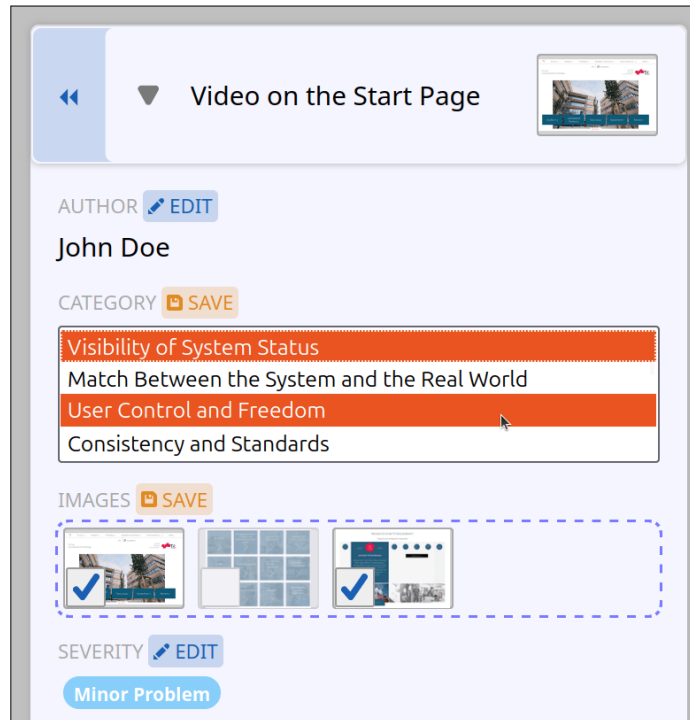


Figure 4.3: Properties can be made editable. A double click replaces the text with a select box.
[Screenshot by Martin Rabensteiner.]

the `<summary>` elements, representing the titles of the tabs. If the titles are too long for the viewport, the titles would break and be represented in more than one line. This is not a suitable solution for tabs and they should be controlled by a select box, as described before. Within the observer, the top offset of the first and last object are compared, and if they are not the same, the responsive solution should be applied. A CSS class (`.compact` in the example) is added to the tab list. The titles can not be simply hidden, as this would give the observer an undetermined width and lever its functionality. Therefore, the titles are made transparent, paddings removed, and the height set to 1px. This makes it practically invisible, but leaves the observer functionality intact.

4.5 Items

Items are created with a `<details>` element. A classname and a `data-role` attribute are added from the predefined selectors. The inner part is defined by a template when initialising. All values that are defined for the item in the `items` object can be used for rendering. Some values are added depending on the current state. These can be:

- `status`: Moved, merged or mergeitem.
- `parent_id`: Contains the item's ID itself again, to be used in nested templates.
- `mergedfrom`: Is a list of IDs from which the item is merged, applies only to mergeitems.
- `mergedinto`: Contains the ID of the mergeitem, applies only to items that are merged.
- `titleimg`: The first image that has an active status that is not negative, applies only to items with images.

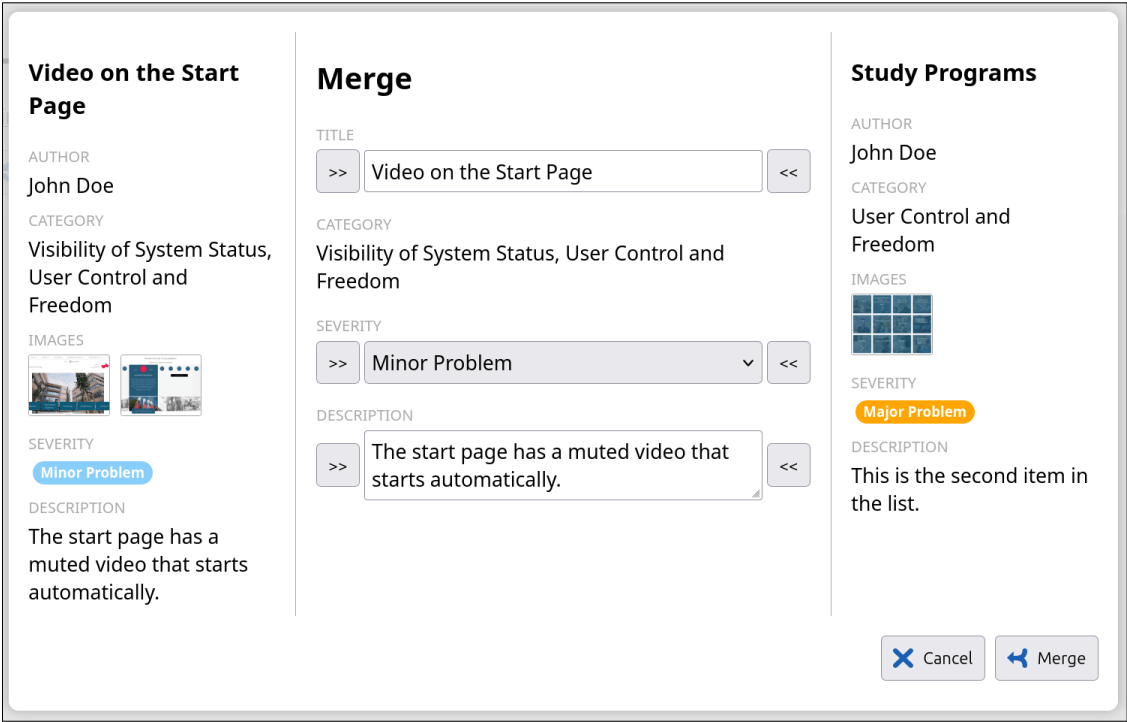


Figure 4.4: When merging two items, their properties are compared in a modal and can be edited in the middle. [Screenshot by Martin Rabensteiner.]

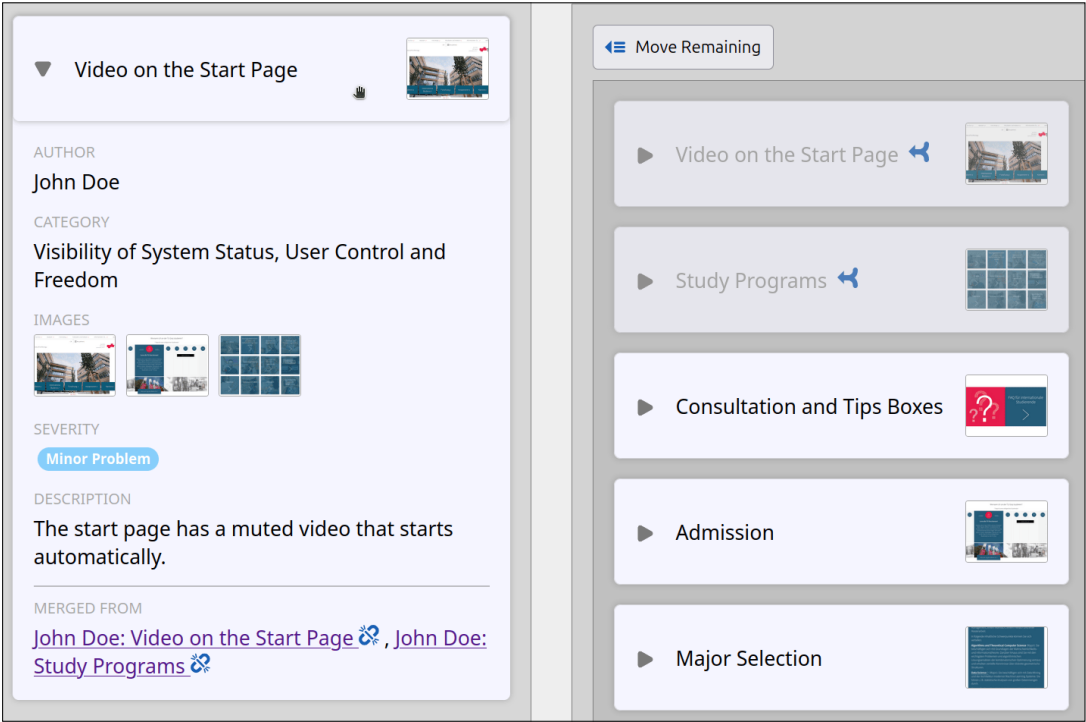


Figure 4.5: Merged items (right) appear greyed out and annotated with a merge icon. The merge item (left) contains links to them. [Screenshot by Martin Rabensteiner.]

4.6 User Actions and their implementations

4.6.1 Moving Items

When items are moved to the mergelist, a clone of the regarding item is produced.

4.6.2 Editing Items

The editing of items is not handed by the library directly, as this can strongly depend on the individual implementation. Therefore, the `events.edit()` function can be called from the outside.

4.6.3 Merging Items

Merging is initiated by dragging one item over another in the mergelist. A dialogue opens to compare the two items to merge, as seen in Figure 4.4. The properties are shown to the left and to the right. The middle column is for setting the properties of the mergeitem. The properties of the left item are taken as default values. Values can also be set or appended by clicking on the buttons left and right to the fields. The items are only merged after clicking on the Merge button. Original items are linked to the merge item and vice versa. These connections can be detached.

4.6.4 Detaching Items

A detach button or link can be added to mergeitems or items that or merged to loosen their connection. All attribute from the mergeitem stay the same, except for the connection to the original item to detach. This item then appears untouched in the origin list again. Two attributes need to be set in order to execute the detach: `data-from` contains the ID of the original item, `data-to` contains the ID of the mergeitem.

4.7 Icons

Especially the merging icon made it necessary to work out custom icons for this project. The icons were initially drafted in Inkscape and the resulting SVG files manually revised and improved. All icons base on a 50x50 viewbox. Strokes have usually a width of 12 and a round linejoin. All icons are listed in Table 4.1. The iconset does not have to be used, but can also be replaced or hidden completely. All items are defined only in CSS stylesheets or templates and therefore not obligatory for the library.

4.8 Challenges and Future Work

This section covers some noteworthy challenges during the development process, that could not completely be satisfied, or just with some workarounds, and further tasks.

4.8.1 Cursor Behaviour when Dragging

One useful feature idea was that the cursor is different when dragging an item, depending on whether the item is approaching to be moved to the mergelist (moving cursor) or merged into another item (adding cursor). The `event.datatransfer.dropEffect` property in JavaScript defines the visual behaviour of the mouse cursor during a drag event [MDN 2025b]. While the `cursor` CSS property offers a variety of 36 mouse cursors, in interaction with the operating system [MDN 2026c], this property is not taken into account as soon as a drag event is started. The `dropEffect` has offers a selection of only four elements, all equivalent to options for the CSS property (`copy`, `move`, `link`, `none`).

Mozilla Firefox does currently not support the change of the cursor, as long as the concerning event listener is not changed. In the current implementation, the event listener is the same for the mergelist and its items. Otherwise, the event listener needs to be reassigned every time an item is changed. Google Chrome does basically support the change during an event, but at some points a `none` cursor is wrongly shown.

Icon	Filename	Description
	caret.svg	Replaces the original triangle for <details>. Indicates and controls its open status. When opened, the icon is rotated 90 degrees.
	check.svg	Used to indicate that an item is added to the mergelist, but not merged.
	close.svg	For close buttons in dialogs.
	detach.svg	To detach items from mergeitems or the other way around.
	dragicon.svg	Used to rearranged items in the mergelist. Not included in the example.
	edit.svg	Button for editing values.
	merge.svg	Button to merge and also as indicator for items that are merged into another.
	move.svg	For moving an item to the mergelist or back.
	moveall.svg	Move all remaining items to the mergelist.
	openfile.svg	Open a file as source for lists and items.
	redo.svg	History redo.
	save.svg	Save the current state of all lists.
	save_orange.svg	Save the current state of an item when editing.
	undo.svg	History undo.

Table 4.1: Overview of icons made for and used within the library.

4.8.2 Flexbox Layout within display: contents

In the examples, the whole page is nested in a flexbox grid [MDN 2025d]. All outer controls, titles, etc. are always visible, and only the lists are scrollable. Due to the construction of tabs with the <details> tag, the CSS option display: contents is used once to let the title be in row and the inner content only for one list. This option shows only the inner part of the content and do not produce a box by themselves [MDN 2026d].

The outer box is removed, the flex bounding box is not valid any more, and therefore the inner content scales to maximum, making the scrollbar for the whole tab scope, and not just the inner part where the items are. Furthermore, accessibility problems associated with this property value are mentioned.

Different structures for HTML and CSS only tabs, like radio buttons and labels [Kliemann 2023] were tried, but led to the same problem. Eventually, a JavaScript ResizeObserver [MDN 2025f] is introduced to monitor the tab height, and adjust the inner lists height to it.

4.8.3 Accessibility

One of the main usability affordance of this project is to have a drag-and-drop functionality. However, this is a functionality that is not in line with accessibility affordance. Some features, like the moving of items can already be done only by clicks, but others, like merging or arranging can not. A possible implementation could be a button for an accessibility, where buttons and menus for those actions are shown additional next to items.

Appendix A

End User Guide

This chapter serves explanatory notes for end users of this library. Note that some procedures and designs described in this chapter may differ for the actual implementation in the specific tool. An example implementation can be seen in Figure A.1 and explanations refer to this architecture.

A.1 Functionality

The main objective of this library is to have a number of lists, the *origin lists* on one side (usually right) and an aggregated list on the other side (left), the *mergelist*. Users can switch between these origin lists via tabs, or on more narrow screens with a select box. Items can be dragged and dropped from one list on the right to the mergelist on the left. An item is inserted at the location where it is dropped. The Move Remaining button can be used to move all existing items of a list to the mergelist. The appearance of items in the list, their extended view, and the merge dialogue are dependent on the individual implementation.

By dragging one item over another in the mergelist, those items can be merged. By dragging items in the mergelist around, they can be rearranged. Undo and Redo let the user jump through the history.

The behaviour of saving is given by the individual implementation and can automatically, or by clicking a Save button. Data is in principle stored in the JSON format. When a saved file is opened again, it must be noted that the file format has the same version as required by the library.

Next to the tab titles can be small numbers to indicate the current state of the list. For origin lists, they count how much of items have been added or merged into the mergelist out of the total number per list. For the mergelist, the indicator shows the total number of items in the list, not necessarily how much have been added.

Properties of items can be edited by clicking on them. Text-only properties are then directly editable and saved as soon as the focus is changed to another element by mouse or keyboard. More complex properties, like select boxes, need a double click to change to editing mode.

A.2 Keyboard Shortcuts

ListMerger offers shortcuts to ease the use with a keyboard. These shortcuts are:

- *Ctrl* + *Z*: Undo.
- *Ctrl* + *Y*: Redo.
- *C*: Collapse all <detail> elements.
- *M*: Move all remaining elements from an origin list to the mergelist.
- *[1-9]*: Switch the current tab of lists. The number indicates their index.

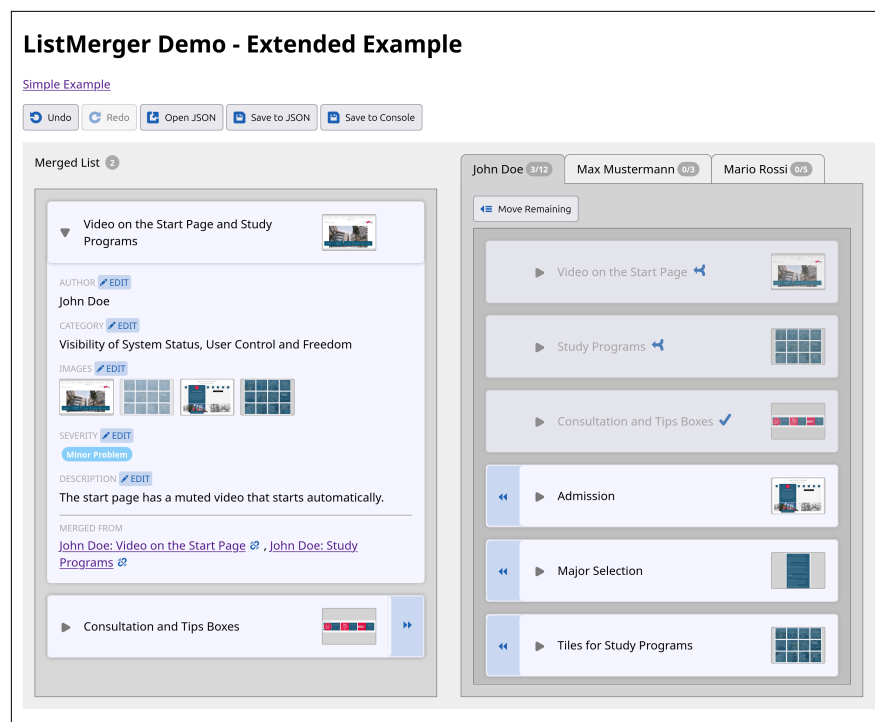


Figure A.1: An example implementation of ListMerger. On the right are [Screenshot by Martin Rabensteiner.]

Appendix B

Library User Guide

This section addresses to library user that want to include the ListMerger library in a larger software project. Some general remarks on the usage of the library are already conducted in Appendix A. This chapter is more focused on the initialisation.

B.1 Examples

Within the code, a simple and an extended example of implementations are delivered and can be found in the `/src/example` folder. The simple example (`simple.html` and `simple.js`) demonstrate a minimal setup. The extended example (`index.html` and `main.js`) is a showcase of nearly all features implemented in the library. To ensure the correct functioning of some elements, certain CSS styling rules are highly recommended and can be found in `listmerger.css`. Further styling recommendations can be found linked in the examples (`style.css`).

B.2 Initialising

To initialise the library within an application, the `init()` function needs to be called. This function has four parameters:

- `items`: A data structure of items and lists, further described in Section B.3
- `templates`: The templates for rendering items, further described in Section B.4.
- `selectors`: Identifiers for HTML class names, IDs, attributes, etc., further described in Section B.5.
- `history_callback`: A function that is called when changes are made.

The parameter `items` is the only mandatory one, the others are optional. When they are left empty, this results in blank templates, standard selectors, respectively an empty function for the callback.

B.3 Data Structure

Items have to be handed over in an object that needs to contain at least a `merged` key and an `originlists` key. The first can be an empty array, or already contain items. The second one needs to be filled with the lists as objects with some metadata (`id`, `name`, etc.) and an `items` key, which also contains an array of items, as seen in Listing B.1.

One item needs to have at least an `id`. This ID is also used as ID in the HTML DOM. The other keys are optional and need to be included depending on whether they are needed in the user-defined templates. Arrays (like images) are always merged together in the merging process.

```

1 {
2   "merged": [],
3   "originlists": [
4     {
5       "id": "eval1",
6       "name": "List 1",
7       "items": [
8         {
9           "id": "lilit1",
10          "title": "Item 1",
11          "author": "Martin Rabensteiner",
12          "category": "Category 1",
13          "images": [
14            "images/screenshot1.png",
15            "images/screenshot2.png"
16          ],
17          "description": "This is the first item in the list."
18        }
19      ]
20    }
21  ]
22 }

```

Listing B.1: Example of an initialising object for items. The items in merged are rendered to the mergelist, originlists are constructed as lists with the base items.

During runtime, the data is stored in the same format on the stack. When finished, it can be suitable to store the data structure as a JSON file, or to send it in this format to a REST API. More details about this are in Section B.6.

B.4 Templating

Templates define how the items appear in the lists. When initialising, the `template` parameters is a string-string pair with up to four keys. These are:

- `"item"`: The content of an item in the list, will be surrounded by a `<details>` tag.
- `"dialog"`: The content shown left and right in a merge dialogue, representing the items to merge.
- `"merge"`: The middle column in merge dialogue, containing form items for the future mergeitem.
- `"draghandle"`: Option to add an element to an item which functions as handle to drag.

An item template contains ideally a `<summary>` tag, so that the expand and collapse functionality is given. In the merge dialogue, the name attribute of form elements need to be identical with the keys used in an item to be recognized correctly.

To show an example, the following snippet for `item` with the data example from Listing B.1:

```

<summary>{{title}}</summary>
{{author}}: {{description}}

{{#images}}
  
{{/images}}

```

renders to:

```
<details id="li1it1">
  <summary>Item 1</summary>
  Martin Rabensteiner: This is the first item in the list.

  
  
</details>
```

B.5 Selectors

The selectors of all HTML class names and IDs can be set during the initialisation. They are used as selectors for events and dynamic changes, but also for stylings. The identifiers, default values, and a short description can be seen in Table B.1.

B.6 Saving

The current state of the list can always be received with the `getAllItems()` function. This returns a JavaScript object in the same format as described in Section B.3, and additionally a `version` field to indicate changes in the schema. The version at release is `v0.1`. For changes, please conduct the changelog in the GitHub repository.

The `history_callback` function can be a helpful tool when the state should always be saved in real time. A function can be handed over during initialisation. The function can have the boolean parameter `is_new`, which is true when a user action has happened, and false when an action was triggered from an undo or redo action.

B.7 Inline Editing

Fields of items can be configured to be editable inline, as seen in Figure B.1. For text-only fields, the `contenteditable` attribute [MDN 2025e] should be set to `"plaintext-only"`. Setting it to `"true"` is also sufficient for the functionality, but can lead to unexpected behaviour when dragging an image over the field.

To use a `<select>` element for editing, the CSS class `selectedit` can be put on a `<span>` element to replace its content. The `<span>` needs to have some `data-*` attributes to build the select box. These attributes are:

- `data-name`: Is used as name for the `<select>` element and for storing to the item, therefore it should be one of the already set children of the item.
- `data-options`: Needs to have the possible options of the select box. It can be either a one-dimensional array, or an object with key-value pairs. If it is an object, the key and values are used as they are, otherwise the single values are used for both.
- `data-values`: Can be a single value, or comma-separated the selected values. Valid examples are:
 - `1,2,3,4`
 - `["John", "Max", "Mario"]`
 - `{"0": "Very Low", "1": "Low", "2": "Medium", "3": "High", "4": "Very High"}`
- `data-mode`: Per default, the select box is displayed as single select. When the mode is `multiple`, the `multiple` attribute is added to the `<select>` element as well. Another option for this attribute is `class`, to change the class of this element to a specific mode, for example to represent colors or icons of the current value.

Identifier	Type	Default Value	Description
PREFIX_MOVED	Prefix for ID	"moved-"	Prefix that is added to the HTML IDs of moved items in the mergelist.
PREFIX_MERGED	Prefix for ID	"merged-"	Prefix that is added to the HTML IDs of merged items in the mergelist.
CONTAINER	ID	"listmerger"	Contains the mergelist and container for origin lists.
MERGE_LIST_CONTAINER	ID	"mergelist-container"	Is a wrapper for the mergelist and its indicator.
MERGE_LIST	ID	"mergelist"	Is the mergelist itself.
LIST	Class	"list"	Every list of items.
ITEM	Class	"item"	A single item within a list.
ITEM_DRAGHANDLE_ACTIVE	Class	"draghandle-active"	The item is currently dragged with the draghandle.
ITEM_DRAG_ACTIVE	Class	"drag-active"	The item is currently dragged itself.
TAB_CONTAINER	Class	"tab-container"	The outer container for the tabs and the tab select box.
TAB_BAR	Class	"tabbar"	The inner container for all tabs.
TAB_NAME	Name attribute	"tabs"	Attribute that is added to the <details> tag of an item.
TAB_COMPACT	Class	"compact"	When there is not enough space for a tab titles, this class is added to the tab bar.
TAB_SELECTOR	ID	"detailsselector"	The tab select box shown in the compact state.
HOVER_DROP	Class	"drop"	The effect added to a list or item, when an item is dragged over it.
ARRANGEBAR	Class	"arrangebar"	The bar shown between two items in the mergelist when rearranging or dragging.
BUTTON_MOVE	Class	"move"	The button to move one item to the mergelist or back from there to the origin list.
BUTTON_MOVEALL	Class	"moveall"	The button to move all remaining items of one origin list to the mergelist.
BUTTON_SAVE	Class	"save"	Save button for the edit dialogue (currently not in use and replaced with inline editing).
BUTTON_CLOSE	Class	"close"	The Button that cancels the merge dialogue.
BUTTON_MERGE	Class	"merge"	The Button that performs the merge after the dialogue.
BUTTON_DETACH	Class	"detach"	Added to merged items and their origin items to unpair the merge.
BUTTON_DRAGHANDLE	Class	"draghandle"	Identifies the element of the drag handle template.
BUTTON_UNDO	ID	"undo"	The undo button for the history.
BUTTON_REDO	ID	"redo"	The redo button for the history.
DIALOG_MERGE_CONTAINER	Class	"mergecontainer"	Container within the merge dialogue.
TEXT_BUTTON_MOVEALL	Text	"Move Remaining"	Shown at every tab.
TEXT_BUTTON_MERGE	Text	"Merge"	Shown in the merge dialogue.
TEXT_BUTTON_CANCEL	Text	"Cancel"	Shown in the merge dialogue.

Table B.1: Identifiers used for HTML, CSS, and JavaScript and their description. They can be set while initialising, or otherwise the default values are used.

The screenshot shows a web form with the following sections:

- Header:** A blue bar with a back arrow, a dropdown menu, and the title "Video on the Start Page".
- AUTHOR:** A label with an "EDIT" button. The name "John Doe" is displayed below.
- CATEGORY:** A label with a "SAVE" button. Below it is a list of categories: "Visibility of System Status", "Match Between the System and the Real World", "User Control and Freedom" (highlighted in orange), and "Consistency and Standards".
- IMAGES:** A label with a "SAVE" button. Below it is a dashed blue box containing three image thumbnails. The first and third thumbnails have a blue checkmark in the bottom-left corner.
- SEVERITY:** A label with an "EDIT" button. Below it is a button labeled "Minor Problem".

Figure B.1: An example of inline editing, which is currently active for Category and Images. The image editing is not part of the library itself but can be taken from the main example. [Screenshot by Martin Rabensteiner.]

- **data-class:** If the mode is set to `class`, this value is used as CSS class prefix. When the selection is changed, the class consisting of the prefix and the selected value is added to this element. Other classes with this prefix are removed on an input change.

B.8 Draghandle

A draghandle can be added to an item to allow only arranging, when dragging the item. This can be useful when the design requires a stronger distinction between merging and arranging items, for example when there is no space between two items or the separating bar between two items should not be shown. Some tags, for example `<img>` have a draggable attribute per default. For other elements it is necessary to add the attribute `draggable="true"`. The snippet below shows a template example with the already included icon:

```

```


Bibliography

- Andrews, Keith [2025]. *Human Computer Interaction: Lecture Notes*. 27 May 2025. <http://courses.isds.tugraz.at/hci/hci.pdf> (cited on page 3).
- Bun [2026]. *Bun: Package Manager*. 23 Jun 2026. <https://bun.com/package-manager> (cited on page 8).
- Cimpanu, Catalin [2019]. *Browser Vendors win War with W3C over HTML and DOM Standards*. 28 May 2019. <https://zdnet.com/article/browser-vendors-win-war-with-w3c-over-html-and-dom-standards/> (cited on page 7).
- Kliemann, Joachim [2023]. *Building a Tab Component with Pure CSS using Radio and Label Tags*. 22 Oct 2023. <https://dev.to/joxx/building-a-tab-component-with-pure-css-using-radio-and-label-tags-200b> (cited on page 15).
- Loitzl, Martin [2006]. *The Heuristic Evaluation Manager (HEM): An Online Collaborative Environment for Heuristic Evaluation*. Master's Thesis. Institute for Information Systems and Computer Media (IICM): Graz University of Technology, Feb 2006. 228 pages. <https://ftp.isds.tugraz.at/pub/theses/mloitzl.pdf> (cited on page 4).
- Maj, Wojciech [2024]. *Package Manager Wars. The Real Picture*. 26 Jan 2024. <https://dev.to/wojtekmaaj/package-manager-wars-the-real-picture-e9p> (cited on page 8).
- Maj, Wojciech [2026]. *Repository: Package Manager Stats*. 13 Mar 2026. <https://github.com/wojtekmaaj/package-manager-stats> (cited on page 8).
- MDN [2025a]. *<details>: The Details Disclosure Element*. 13 Oct 2025. <https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/details> (cited on page 8).
- MDN [2025b]. *DataTransfer: dropEffect property*. 02 Oct 2025. <https://developer.mozilla.org/en-US/docs/Web/API/DataTransfer/dropEffect> (cited on page 14).
- MDN [2025c]. *Document: querySelector() Method*. 07 Nov 2025. <https://developer.mozilla.org/en-US/docs/Web/API/Document/querySelector> (cited on page 8).
- MDN [2025d]. *Flexbox: One-Dimensional Layout Method for Arranging Items in Rows or Columns*. 19 Dec 2025. https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/CSS_layout/Flexbox (cited on page 15).
- MDN [2025e]. *HTMLElement: contentEditable Property*. 10 Apr 2025. <https://developer.mozilla.org/en-US/docs/Web/API/HTMLElement/contentEditable> (cited on page 21).
- MDN [2025f]. *ResizeObserver: Reports Changes to the Dimensions of an Element's Content*. 07 Nov 2025. <https://developer.mozilla.org/en-US/docs/Web/API/ResizeObserver> (cited on pages 11, 15).
- MDN [2026a]. *CSS: Color()*. 07 Nov 2026. https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Values/color_value/color (cited on page 7).
- MDN [2026b]. *CSS: Container Queries*. 24 Mar 2026. https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Containment/Container_queries (cited on page 7).

- MDN [2026c]. *CSS: Cursor*. 07 Jan 2026. <https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties/cursor> (cited on page 14).
- MDN [2026d]. *CSS: Display Property*. 20 Apr 2026. <https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties/display> (cited on page 15).
- Mustache [2024]. *Mustache: Logic-Less Templates*. 09 Aug 2024. <https://mustache.github.io/> (cited on pages 8–9).
- Nielsen, Jakob and Rolf Molich [1990]. *Heuristic Evaluation of User Interfaces*. Proc. SIGCHI Conference on Human Factors in Computing Systems (CHI 1990) (Seattle, Washington, USA). ACM. 01 Apr 1990, pages 249–256. doi:10.1145/97243.97281. <https://dl.acm.org/doi/pdf/10.1145/97243.97281> (cited on page 3).
- Node.js [2026]. *Node.js: Run JavaScript Everywhere*. 23 Jun 2026. <https://nodejs.org/> (cited on page 7).
- NPM [2026]. *NPM Documentation*. 19 Mar 2026. <https://docs.npmjs.com/> (cited on page 8).
- PNPM [2026]. *PNPM: Fast, Disk Space Efficient Package Manager*. 19 Mar 2026. <https://pnpm.io/> (cited on page 8).
- Shukla, Abhishek [2023]. *Modern JavaScript Frameworks and JavaScript’s Future as a Full-Stack Programming Language*. Journal of Artificial Intelligence & Cloud Computing 144.2 (Oct 2023), pages 2–5. doi:10.47363/JAICC/2023(2)144. <https://onlinescientificsresearch.com/articles/modern-javascript-frameworks-and-javascripts-future-as-a-full-stack-programming-language.pdf> (cited on page 7).
- W3C [2026]. *Cascading Style Sheets*. 14 Apr 2026. <https://www.w3.org/Style/CSS/> (cited on page 7).
- W3Techs [2026]. *Historical Yearly Trends in the Usage Statistics of JavaScript Libraries for Websites*. 20 Mar 2026. https://w3techs.com/technologies/history_overview/javascript_library/all/y (cited on page 8).
- WHATWG [2026]. *HTML - Living Standard*. 14 Apr 2026. <https://html.spec.whatwg.org/> (cited on page 7).